ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO THỰC HÀNH

MÔN HỌC: HỆ ĐIỀU HÀNH (CS333)

LAB 3
PAGE TABLES

Sinh viên thực hiện:	Võ Nhất Bảo	24120166
	Phạm Lê Huy Hoàng	24120182
Mã nhóm:	24120166_24120182	

Thành phố Hồ Chí Minh, tháng 11 năm 2026

Mục lục

1 Thông tin nhóm và phân công công việc 2
2 Môi trường và kiến thức nền 2
3 Task 1: Inspect a user-process page table 3
4 Task 2: Walk and print a page table 4
5 Task 3: Detect which pages have been accessed 5
6 Task 4: Speed up the getpid system call 7
7 Task 5: Use superpages 8
8 Kiểm thử 9
9 Thảo luận và tự đánh giá 10
10 Tài liệu tham khảo 10

1. Thông tin nhóm và phân công công việc

STT	MSSV	Họ và tên	Công việc đảm nhận	Đóng góp
1	24120166	Võ Nhất Bảo	Task 1: giải mã page table của `pgtbltest` (Q1). Task 2: `vmprint()` và Q2. Task 3: system call `pgaccess()`, định nghĩa `PTE_A`, chương trình kiểm thử bổ sung `user/pgaudit.c`. Ảnh minh chứng và các mục 1 đến 5 của báo cáo.	50%
2	24120182	Phạm Lê Huy Hoàng	Task 4: trang `USYSCALL` và `ugetpid()`, Q3. Task 5: superpage cho kernel page table, dấu superpage trong `vmprint`, `kpgstat`, Q4. Thí nghiệm E1–E8, mục 6 và 7.	50%
Công việc chung			Chạy autograder, rà soát chéo mã nguồn, `time.txt`, báo cáo, đóng gói bài nộp.	Cả hai

Bảng 1. Phân công công việc và tỷ lệ đóng góp.

Task	Nội dung	Tệp chính	Hoàn thành	Autograder
1	Đọc page table của process (Q1)	`answers-pgtbl.txt`	100%	chấm tay
2	`vmprint` (Q2)	`kernel/vm.c`, `kernel/sysproc.c`	100%	10/10
3	`pgaccess`	`kernel/riscv.h`, `kernel/sysproc.c`, `user/pgaudit.c`, `Makefile`	100%	10/10
4	Trang `USYSCALL` (Q3)	`kernel/proc.c`, `kernel/proc.h`	100%	10/10
5	Superpage (Q4)	`kernel/vm.c`, `kernel/main.c`, `user/kpgstat.c`, `Makefile`	100%	10/10
	Câu trả lời, số giờ	`answers-pgtbl.txt`, `time.txt` (10 giờ)	100%	5/5, 1/1
Tổng `make grade`				**46/46**

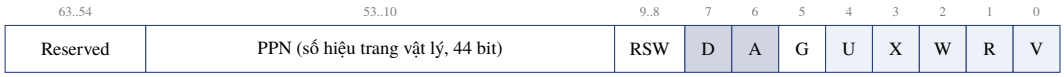
Bảng 2. Mức hoàn thành tự đánh giá và điểm của `make grade`. Test `answers-pgtbl.txt` chỉ kiểm tra tệp có nội dung; câu trả lời được chấm tay.

2. Môi trường và kiến thức nền

Môi trường. NixOS 26.05 (x86-64). `shell.nix` của nhóm cung cấp `riscv64-none-elf-gcc` 15.3.0, Binutils 2.46, QEMU 10.2.4 (3 CPU), Python 3 và `pexpect`. Mã nguồn lấy từ `git://g.csail.mit.edu/xv6-labs-2026`, nhánh `pgtbl`, commit gốc `d1eb62f`, `conf/lab.mk` đặt `LAB=pgtbl`. Nhóm không sửa `grade-lab-pgtbl`, `gradelib.py` và `user/pgtbltest.c`. Ảnh chụp trong báo cáo do script `scripts/capture_lab3.py` (nằm ngoài repo) tạo: script chạy xv6 hoặc `make grade` thật trong xterm rồi chụp cửa sổ, log văn bản lưu cạnh ảnh.

Task 4, 5 được làm và kiểm tra trên WSL Ubuntu 24.04 (gcc 13.3.0, QEMU 8.2.2); Hình 5–7 được render từ log thật của máy này. Ở đó text của `pgtbltest` dài hai trang nên các va thấp lệch một trang so với Hình 2.

Sv39. Xv6 dùng sơ đồ Sv39 của RISC-V [3][4]. Địa chỉ ảo dùng 39 bit: ba chỉ số 9 bit (bit 38 đến 30, 29 đến 21, 20 đến 12) chọn PTE ở ba mức page table, 12 bit cuối là offset trong trang. Mỗi trang page table chứa 512 PTE 64 bit. Một PTE có `V=1` và `R=W=X=0` trỏ xuống trang page table mức dưới; PTE có ít nhất một trong `R`, `W`, `X` là PTE lá. Lá ở mức 0 map một trang 4 KB, lá ở mức 1 map 2 MB (megapage, dùng ở Task 5). Xv6 chỉ dùng nửa dưới không gian địa chỉ: `MAXVA` $= 2^{38} =$ `0x4000000000`.



Hình 1. Cấu trúc PTE Sv39. `PTE2PA(pte) = (pte >> 10) << 12`, `PTE_FLAGS(pte) = pte & 0x3FF`. Phần cứng đặt `A` khi dùng PTE cho bất kỳ truy cập nào và đặt `D` khi dùng nó cho lệnh ghi.

3. Task 1: Inspect a user-process page table

`print_pgtbl()` trong `pgtbltest` gọi system call `pgpte(va)` cho 10 trang đầu và 10 trang cuối dưới `MAXVA`. Phía kernel (`kernel/stats.c`), `sys_pgpte()` gọi `walk(p->pagetable, va, 0)` và trả về PTE mức 0, hoặc 0 nếu không có. Chương trình in `pte`, `pa = PTE2PA(pte)` và `perm = PTE_FLAGS(pte)`, gồm cả bit `A`, `D`.

Để biết trang nào chứa gì, nhóm xem program header của `user/_pgtbltest` bằng `riscv64-none-elf-readelf -l`. Segment thứ nhất (`.text` và `.rodata`, 0xfb1 byte, quyền `R E`) nằm ở va 0. Segment thứ hai (`.data` và `.bss`, 0x30 byte, `RW`) nằm ở `0x1000` vì `user/user.ld` căn `.data` theo trang. Sau đó `exec()` cấp thêm `USERSTACK + 1 = 2` trang cho guard page và stack, nên `p->sz = 0x4000`. Ví dụ trong đề có hai trang text; với trình biên dịch của nhóm, text và rodata vừa một trang.

```
$ pgtbltest
print_pgtbl starting
va 0x0 pte 0x21FC885B pa 0x87F22000 perm 0x5B
va 0x1000 pte 0x21FC7C17 pa 0x87F1F000 perm 0x17
va 0x2000 pte 0x21FC7807 pa 0x87F1E000 perm 0x7
va 0x3000 pte 0x21FC74D7 pa 0x87F1D000 perm 0xD7
va 0x4000 pte 0x0 pa 0x0 perm 0x0
va 0x5000 pte 0x0 pa 0x0 perm 0x0
va 0x6000 pte 0x0 pa 0x0 perm 0x0
va 0x7000 pte 0x0 pa 0x0 perm 0x0
va 0x8000 pte 0x0 pa 0x0 perm 0x0
va 0x9000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFF6000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFF7000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFF8000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFF9000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFA000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFB000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFC000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFD000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFE000 pte 0x21FD08C7 pa 0x87F42000 perm 0xC7
va 0x3FFFFFF000 pte 0x20001C4B pa 0x80007000 perm 0x4B
print_pgtbl: OK
```

Dòng `va 0x0 pte 0x21FC885B pa 0x87F22000 perm 0x5B`:

- `pa` = `(0x21FC885B >> 10) << 12` = `0x87F22000`;
- `perm` = `0x5B` = $0101\,1011_2$: bit 0 (V), 1 (R), 3 (X), 4 (U), 6 (A).

Các dòng khác giải mã theo cùng cách ở Bảng 3. Bit `G` và hai bit `RSW` bằng 0 ở mọi dòng. Kernel ghi vào trang user qua direct map (`loadseg` trong `exec`, `copyout`), không đi qua PTE của user, nên những lần ghi đó không bật `A` hay `D`.

Hình 2. Phần `print_pgtbl` trong output thật của `pgtbltest`, chụp trước khi có Task 4.

Q1. Mỗi PTE trong output của `print_pgtbl` chứa gì và các bit quyền là gì?

va	perm	Bit	Nội dung	Giải thích
`0x0`	`0x5B`	V R X U A	Text và rodata	Mã của `pgtbltest`, `printf`, `malloc`, `ulib` và hằng số (bảng `digits` của `printf`). User đọc và thực thi được; `W=0` nên lệnh ghi gây page fault, chương trình không tự sửa mã của mình. `A=1` vì CPU đã fetch lệnh, `D=0` vì trang chưa bị ghi.
`0x1000`	`0x17`	V R W U	Data và bss	Biến toàn cục `testname`, `freep` và `base` của `malloc`. Đọc và ghi được, `X=0` nên dữ liệu không chạy được như mã. `A=D=0`: lúc in, chương trình chưa dùng biến nào trong số này (`testname` được gán ở `pgaccess_test`, `freep` ở lần `malloc` đầu tiên).
`0x2000`	`0x07`	V R W	Guard page	`exec()` cấp trang này như trang thường rồi `uvmclear()` xóa `PTE_U`. User truy cập là page fault, nên stack tràn thì process bị kill thay vì ghi đè lên trang data. `walkaddr()` cũng từ chối trang thiếu `PTE_U`, nên `copyin/copyout` không dùng được nó.
`0x3000`	`0xD7`	V R W U A D	User stack	Một trang (`USERSTACK = 1`); `exec()` đặt chuỗi `argv` và mảng `argv` ở đỉnh, `sp` đi xuống dần về guard page. `A=D=1` vì mỗi lời gọi hàm đọc và ghi stack frame.
`0x4000` đến `0x9000` (6 dòng)	`0x0`	V=0	Chưa map	Heap sẽ mọc từ `p->sz = 0x4000` khi gọi `sbrk`; lúc này `pgtbltest` chưa `malloc`. Slot PTE có tồn tại (cùng trang page table mức 0 phủ 0 đến `0x1FFFFF`) nhưng bằng 0. Truy cập vào đây gây page fault và `vmfault()` không cấp trang vì va $\geq$ `p->sz`.
`0x3FFFFF6000` đến `0x3FFFFFC000` (7 dòng)	`0x0`	V=0	Chưa map	Khoảng trống giữa heap và các trang ở đỉnh không gian địa chỉ. Kernel stack không nằm ở đây; chúng chỉ có trong kernel page table.
`0x3FFFFFD000`	`0x0`	V=0	`USYSCALL`	`USYSCALL = TRAPFRAME - PGSIZE`. Chưa có Task 4 nên trang chưa được map, và `ugetpid_test` lỗi với `stval=0x3fffffd000`. Task 4 map trang này với `V|R|U` (`0x13`, thêm `A` khi đã đọc), chứa `struct usyscall`.
`0x3FFFFFE000`	`0xC7`	V R W A D	Trapframe	`p->trapframe`. `U=0`: user không đọc, không sửa được. Chỉ mã trampoline ở supervisor mode dùng nó khi `satp` còn là page table của user: `uservec` lưu 31 thanh ghi vào đây, `userret` nạp lại; vì vậy `A=D=1`. Trang vật lý do `allocproc()` cấp lúc `fork`, trước `exec` khá lâu, nên địa chỉ nằm xa các trang khác.
`0x3FFFFFF000`	`0x4B`	V R X A	Trampoline	`uservec`/`userret`. `pa 0x80007000` là symbol `trampoline` trong text của kernel (`kernel/kernel.sym`). Trang này được map ở cùng địa chỉ ảo trong kernel page table và trong mọi user page table, nên mã vẫn chạy tiếp khi đổi `satp`. Có `R`, `X`, không có `W`; `U=0` nên chỉ supervisor thực thi được. `A=1` do fetch lệnh.

Bảng 3. Giải mã 20 dòng của `print_pgtbl`. Các dòng có `pte 0x0` được gộp theo vùng.

Địa chỉ vật lý không liên tiếp. `kalloc()` lấy trang từ một free list kiểu LIFO mà `freerange()` đã đẩy vào theo thứ tự địa chỉ tăng dần, nên trang được cấp từ cuối RAM đi xuống: `0x87F22000`, `0x87F1F000`, `0x87F1E000`, `0x87F1D000`. Hai trang `0x87F21000` và `0x87F20000` nằm giữa text và data là trang page table mức 1 và mức 0 mà `walk()` cấp khi map trang text; chúng xuất hiện dưới dạng PTE không phải lá trong output của `vmprint` (mục 4).

4. Task 2: Walk and print a page table

4.1. Thiết kế

`vmprint()` in dòng tiêu đề rồi gọi hàm đệ quy `vmprint_level(pagetable, level, va)` cho trang gốc (mức 2, va 0). Hàm này duyệt 512 PTE theo thứ tự chỉ số, giống `freewalk()`:

- PTE có `V=0` bị bỏ qua, đúng yêu cầu không in PTE không hợp lệ.
- Địa chỉ ảo của PTE thứ `i` là `va + (i << PXSHIFT(level))`: mỗi PTE phủ 1 GB ở mức 2, 2 MB ở mức 1 và 4 KB ở mức 0. Với `i = 255` ở mức gốc ta được `0x3fc0000000`, khớp với regex của grader.
- PTE lá (có `R`, `W` hoặc `X`) được in kèm quyền theo thứ tự `R W X U`. PTE còn lại trỏ tới trang page table mức dưới nên hàm đệ quy vào đó với `level - 1`.

Hàm nhận ra lá bằng các bit `R`, `W`, `X` ở mọi mức. Với superpage của Task 5 (lá ở mức 1), hàm in một dòng có quyền và không đệ quy vào vùng nhớ 2 MB như thể đó là page table, nên Task 5 chỉ cần thêm dấu hiệu superpage. Mỗi dòng được in bằng một lời gọi `printk`, nên output của CPU khác không chen vào giữa dòng. Phần thụt lề lấy từ bảng `indent[level]`, mỗi mức sâu thêm một chuỗi `" .."`.

```
static void
vmprint_level(pagetable_t pagetable, int level, uint64 va)
{
  static char *indent[] = {" .. .. ..", " .. ..", " .."};
  for (int i = 0; i < 512; i++) {
    pte_t pte = pagetable[i];
    if ((pte & PTE_V) == 0)
      continue;
    uint64 a = va + ((uint64)i << PXSHIFT(level)); // 1 GB, 2 MB or 4 KB
    if (pte & (PTE_R | PTE_W | PTE_X)) {          // a leaf
      char perm[5], *f = perm;                     // "RWXU" subset
      if (pte & PTE_R) *f++ = 'R';   /* ... W, X, U the same way ... */
      *f = 0;
      printk("%s%p: pte %p pa %p %s\n", indent[level], (void *)a, (void *)pte,
             (void *)PTE2PA(pte), perm);
    } else {                                       // next-level table
      printk("%s%p: pte %p pa %p\n", indent[level], (void *)a, (void *)pte,
             (void *)PTE2PA(pte));
      if (level > 0)
        vmprint_level((pagetable_t)PTE2PA(pte), level - 1, a);
    }
  }
}
```

kernel/vm.c (rút gọn phần dựng chuỗi quyền)

Hai chi tiết tìm ra khi chạy thử. Trong PDF của đề, dấu cách đầu dòng bị mất, nhưng `grade-lab-pgtbl` so khớp `"^ \.\.0x..."`, tức mỗi dòng PTE phải bắt đầu bằng `" .."`. Ngoài ra `printk` được khai báo với thuộc tính `format(printf)`, nên truyền `uint64` cho `%p` bị `-Werror=format` chặn; các đối số được ép sang `void *`. Nhóm cũng đổi kiểu trả về của `sys_vmprint` và `sys_pgaccess` từ `int` sang `uint64` cho khớp với khai báo `extern uint64` trong `kernel/syscall.c`.

```
vmprint_test starting
page table 0x0000000087f25000
 ..0x0000000000000000: pte 0x0000000021fc8401 pa 0x0000000087f21000
 .. ..0x0000000000000000: pte 0x0000000021fc8001 pa 0x0000000087f20000
 .. .. ..0x0000000000000000: pte 0x0000000021fc885b pa 0x0000000087f22000 RXU
 .. .. ..0x0000000000001000: pte 0x0000000021fc7c17 pa 0x0000000087f1f000 RWU
 .. .. ..0x0000000000002000: pte 0x0000000021fc7807 pa 0x0000000087f1e000 RW
 .. .. ..0x0000000000003000: pte 0x0000000021fc74d7 pa 0x0000000087f1d000 RWU
 ..0x0000003fc0000000: pte 0x0000000021fc9001 pa 0x0000000087f24000
 .. ..0x0000003fffe00000: pte 0x0000000021fc8c01 pa 0x0000000087f23000
 .. .. ..0x0000003fffffe000: pte 0x0000000021fd08c7 pa 0x0000000087f42000 RW
 .. .. ..0x0000003ffffff000: pte 0x0000000020001c4b pa 0x0000000080007000 RX
vmprint_test: done
pgaccess_test starting
pgaccess_test: OK
ugetpid_test starting
usertrap(): unexpected scause 0xd pid=3
            sepc=0x6e0 stval=0x3fffffd000
```

Hình 3. Output thật của `vmprint_test`, `pgaccess_test` và lỗi của `ugetpid_test` (Task 4 chưa làm).

4.2. Q2. Các trang lá trong output của `vmprint`

Cây có hai nhánh từ trang gốc `0x87f25000`. Chỉ số 0 trỏ tới trang mức 1 `0x87f21000`, chỉ số 0 của trang này trỏ tới trang mức 0 `0x87f20000` chứa 4 lá cho va `0x0` đến `0x3fff`. Chỉ số 255 trỏ tới trang mức 1 `0x87f24000`, chỉ số 511 trỏ tới trang mức 0 `0x87f23000` chứa 2 lá ở chỉ số 510 và 511, vì `0x3fffffe000` = `255<<30 | 511<<21 | 510<<12`. Các PTE không phải lá có cờ `0x01`, chỉ có `V`.

va	Quyền	Nội dung	Cho phép	Không cho phép
`0x0000000000000000`	RXU	Text, rodata	User đọc và fetch lệnh	Ghi (store page fault), nên mã và hằng số không bị sửa
`0x0000000000001000`	RWU	Data, bss	User đọc và ghi	Thực thi (`X=0`), nên byte ghi vào dữ liệu toàn cục không chạy được như mã
`0x0000000000002000`	RW	Guard page	Không gì cả với user, vì thiếu `U`	Mọi lần đọc, ghi, fetch của user; tràn stack bị bắt tại đây thay vì đè lên data
`0x0000000000003000`	RWU	User stack	User đọc và ghi	Thực thi
`0x0000003fffffe000`	RW	Trapframe	Supervisor đọc và ghi	Mọi truy cập của user (`U=0`), nên user không giả được giá trị thanh ghi mà kernel sẽ nạp lại; thực thi
`0x0000003ffffff000`	RX	Trampoline	Supervisor đọc và thực thi	Ghi; user nhảy vào (`U=0`)

Bảng 4. Sáu trang lá trong output của `vmprint`.

Liên hệ với Task 1. Hai chương trình đọc cùng một page table của cùng process, nên mỗi lá của `vmprint` có đúng `pte` và `pa` của một trong sáu dòng khác 0 của `print_pgtbl` (va `0x0`, `0x1000`, `0x2000`, `0x3000`, `0x3FFFFFE000`, `0x3FFFFFF000`). Mười bốn dòng còn lại của `print_pgtbl` có `pte 0`, là các PTE không hợp lệ mà `vmprint` bỏ qua. `print_pgtbl` chỉ lấy mẫu 20 địa chỉ và chỉ thấy PTE mức 0, nhưng in đủ 10 bit cờ, kể cả `A` và `D`. `vmprint` đi hết cây nên thấy thêm 4 PTE trỏ tới trang page table, và chỉ giải mã `R`, `W`, `X`, `U`. So với Hình 3.4 của sách xv6 [3], process này có đủ text, data, guard page, stack, trapframe, trampoline nhưng chưa có trang heap nào. Sau Task 4, `vmprint` có thêm lá `0x0000003fffffd000` với quyền `RU`: trang `USYSCALL`, user chỉ đọc được.

5. Task 3: Detect which pages have been accessed

5.1. Thiết kế

`pgaccess(base, npages, mask)` nhận địa chỉ đầu, số trang và địa chỉ buffer kết quả. Bit `i` của bitmask (byte `i/8`, bit `i%8`) ứng với trang thứ `i`, trang đầu là bit thấp nhất. `PTE_A` là bit 6 theo đặc tả Sv39 [4]; nhóm định nghĩa nó trong `kernel/riscv.h` cùng `PTE_D` (bit 7). `sys_pgaccess()` làm như sau:

1. Đọc đối số bằng `argaddr/argint`. Từ chối `npages` ≤ 0 hoặc lớn hơn `PGACCESS_MAX = 4096`. Làm tròn `base` xuống đầu trang, vì con trỏ `malloc` trả về không căn trang (nằm sau header 16 byte). Từ chối khoảng địa chỉ vượt `MAXVA`, vì `walk()` gọi `panic` khi `va >= MAXVA`.
2. Lượt 1: với mỗi trang, `walk(p->pagetable, va, 0)` phải trả về PTE có `V` và `U`; thiếu một trong hai thì trả -1. Điều kiện `U` loại guard page, trapframe và trampoline. Nếu PTE có `A` thì bật bit tương ứng trong mảng `mask` trên kernel stack. Lượt này chưa sửa gì.
3. Lượt 2: `copyout` đúng `(npages + 7) / 8` byte ra buffer user. Test truyền `&abits` kiểu `unsigned int` (4 byte) cho 32 trang, nên ghi một `uint64` sẽ đè lên stack của user. `copyout` trả lỗi nếu buffer không được map, nằm trên `p->sz`, ở địa chỉ kernel, hoặc thuộc trang chỉ đọc.

4. Lượt 3: chỉ khi `copyout` thành công mới xóa `PTE_A` của các trang đã báo, nên lần gọi sau chỉ thấy truy cập mới. Một lần gọi thất bại không làm mất thông tin.

```
  if (npages <= 0 || npages > PGACCESS_MAX)
    return -1;
  base = PGROUNDDOWN(base);
  if (base >= MAXVA || npages > (MAXVA - base) / PGSIZE)  // walk() panics
    return -1;
  int nbytes = (npages + 7) / 8;
  memset(mask, 0, nbytes);
  for (int i = 0; i < npages; i++) {                      // pass 1
    pte_t *pte = walk(p->pagetable, base + (uint64)i * PGSIZE, 0);
    if (pte == 0 || (*pte & PTE_V) == 0 || (*pte & PTE_U) == 0)
      return -1;
    if (*pte & PTE_A)
      mask[i / 8] |= 1 << (i % 8);
  }
  if (copyout(p->pagetable, p->sz, umask, (char *)mask, nbytes) < 0)
    return -1;                                            // pass 2
  for (int i = 0; i < npages; i++) {                      // pass 3
    if (mask[i / 8] & (1 << (i % 8))) {
      pte_t *pte = walk(p->pagetable, base + (uint64)i * PGSIZE, 0);
      *pte &= ~PTE_A;
    }
  }
  return 0;
```

kernel/sysproc.c, sys_pgaccess() (bỏ phần đọc đối số)

Giới hạn 4096 trang. Bitmask cho 4096 trang (16 MB) là 512 byte, đặt được trên kernel stack 4 KB. Nhóm chọn đúng 4096 vì `pgaccess_test` gọi hai lần với 4096 trang để kiểm tra trang chưa map và buffer sai; với giới hạn nhỏ hơn, hai lời gọi đó sẽ thất bại vì giới hạn và nhánh kiểm tra thật không được chạy. Lời gọi với 10 000 000 trang bị chặn bởi giới hạn.

TLB. Sau khi xóa `PTE_A`, TLB có thể vẫn giữ bản dịch cũ, và phần cứng chỉ bật lại `A` khi đi lại page table. Không cần gọi `sfence.vma` trong `sys_pgaccess`: `userret` trong `kernel/trampoline.S` đã chạy `sfence.vma` trước và sau khi nạp lại `satp` của user, nên lần truy cập đầu tiên sau system call đi lại page table và bật lại `A`.

5.2. Kiểm thử bổ sung

`pgtbltest` không kiểm tra hết các yêu cầu của rubric, nên nhóm viết `user/pgaudit.c` (thêm vào `UPROGS` trong `Makefile`) với 23 ca: `npages` bằng 0, âm, bằng 4096, bằng 4097; thứ tự bit và số byte được ghi (9 trang ghi đúng 2 byte, byte thứ ba giữ giá trị canh); đọc cũng bật `A`; bit được xóa sau khi báo; `base` không căn trang; buffer ở trang text chỉ đọc, ở địa chỉ kernel, trên `p->sz`; khoảng trang vượt cuối heap; bit `A` còn nguyên sau các lần gọi lỗi; trang `sbrklazy` chưa chạm tới và sau khi chạm; guard page, trapframe, trampoline; khoảng vượt `MAXVA` và va $= 2^{64} - 1$ (kernel không panic). Cả 23 ca đạt (Hình 4).

Để chắc các ca này phát hiện được lỗi thật, nhóm build tạm hai phiên bản sai của kernel rồi khôi phục bản đúng:

- Xóa `PTE_A` ngay ở lượt 1, trước `copyout`. `pgtbltest` vẫn báo `pgaccess_test: OK`, còn `pgaudit` báo lỗi ở ca "A bits survive the failed calls".
- Bỏ lượt 3, không xóa `PTE_A`. `pgtbltest` báo `incorrect access bits set`, `pgaudit` báo lỗi 2 ca.

Trang `sbrklazy` chưa được chạm tới không có PTE, nên `pgaccess` trả -1 theo đúng câu chữ của đề (trang chưa map). Một bộ thu gom rác có thể muốn coi trang đó là "chưa truy cập" (bit 0); nhóm giữ cách hiểu của đề.

```
$ pgaudit
ok   npages = 0 is rejected
ok   npages < 0 is rejected
ok   pgaccess on 9 mapped pages
ok   byte 0 = 0x09: pages 0 and 3, LSB first
ok   byte 1 = 0x01: page 8
ok   only (9+7)/8 = 2 bytes are written
ok   the A bits were cleared by the previous call
ok   unaligned base is rounded down to its page
ok   mask in read-only text is rejected
ok   mask at a kernel address is rejected
ok   mask above p->sz is rejected
ok   range running past the heap end is rejected
ok   A bits survive the failed calls
ok   unmapped page above p->sz
ok   lazy sbrk page not touched yet
ok   lazy page after the first touch
ok   stack guard page (no PTE_U)
ok   trapframe (no PTE_U)
ok   trampoline (no PTE_U)
ok   range past MAXVA (no kernel panic)
ok   va = 2^64 - 1 (no kernel panic)
ok   npages = 4096 (the limit) works
ok   npages = 4097 is rejected
pgaudit: all 23 checks passed
```

Hình 4. Output thật của `pgaudit`.

6. Task 4: Speed up the getpid system call

6.1. Thiết kế

`ugetpid()` (có sẵn trong `user/ulib.c`) chỉ đọc `((struct usyscall *)USYSCALL)->pid`: một lệnh load, không vào kernel. Kernel đặt sẵn trang chứa pid ở `USYSCALL` trong page table của mọi process; theo gợi ý của đề, trang đi theo đúng vòng đời của trapframe, struct proc có thêm trường `usyscall` (`kernel/proc.h`).

```
// allocproc(): allocate the page, clear it, store the pid
if ((p->usyscall = (struct usyscall *)kalloc()) == 0) {
  freeproc(p); release(&p->lock); return 0;          // like the trapframe
}
memset(p->usyscall, 0, PGSIZE);
p->usyscall->pid = p->pid;
// proc_pagetable(): map it below TRAPFRAME, readable by user code only
if (mappages(pagetable, USYSCALL, PGSIZE, (uint64)(p->usyscall), PTE_R | PTE_U) < 0)
  { /* unmap TRAPFRAME and TRAMPOLINE, uvmfree(pagetable, 0), return 0 */ }
uvmunmap(pagetable, USYSCALL, 1, 0);                 // proc_freepagetable(): do_free = 0
if (p->usyscall) kfree((void *)p->usyscall);         // freeproc()
```

kernel/proc.c (rút gọn; các đoạn nằm trong #ifdef LAB_PGTBL)

- **Quyền `PTE_R | PTE_U`** (`perm 0x13`): không có `W`, `X`, nên user ghi vào trang là store page fault.
- **Unmap mà không free.** `exec()` tạo page table mới bằng `proc_pagetable(p)`, map lại đúng `p->usyscall`, rồi mới `proc_freepagetable()` page table cũ; free ở đây thì page table mới trỏ vào trang đã free (E3, Bảng 7). Trang thuộc về struct proc, nên `freeproc()` free nó.
- **Ghi pid một lần**: pid không đổi trong đời process (`exec()` giữ struct proc, `fork()` gọi `allocproc()` cho con). User đọc được cả trang, nên `memset` cho phần còn lại giá trị xác định.

```
$ pgtbltest
[...]
va 0x3FFFFFD000 pte 0x21FEC413 pa 0x87FB1000 perm 0x13
va 0x3FFFFFE000 pte 0x21FE7CC7 pa 0x87F9F000 perm 0xC7
va 0x3FFFFFF000 pte 0x20001C4B pa 0x80007000 perm 0x4B
[...]
 .. .. ..0x0000003fffffd000: pte 0x0000000021fec413 pa 0x0000000087fb1000 RU
 .. .. ..0x0000003fffffe000: pte 0x0000000021fe7cc7 pa 0x0000000087f9f000 RW
 .. .. ..0x0000003ffffff000: pte 0x0000000020001c4b pa 0x0000000080007000 RX
[...]
ugetpid_test starting
ugetpid_test: OK
ksuper_test: OK
pgtbltest: all tests succeeded
```

Hình 5. Output của `pgtbltest` sau Task 4 (trích, `[...]` là dòng được lược). `USYSCALL` có `perm 0x13` = `V R U` và một lá `RU` trong `vmprint`, đúng như dự đoán ở Q1, Q2. `ugetpid_test` so `getpid()` với `ugetpid()` ở process cha và 64 process con.

Không rò trang. `usertests -q` đếm trang trống trước và sau khi chạy (`countfree()`) và báo `lost some free pages` nếu thiếu; với Task 4 nó in `ALL TESTS PASSED`. Bỏ `kfree(p->usyscall)` thì chính kiểm tra này báo lỗi (E1).

6.2. Q3. System call nào khác có thể nhanh hơn nhờ trang dùng chung?

Chỉ những lời gọi *chỉ đọc* một giá trị mà kernel giữ được bản sao luôn đúng trong trang. `fork`, `exit`, `wait`, `kill`, `read`, `write`, `open`, `exec`, `pipe`, `sbrk(n)` với $n \neq 0$ đổi trạng thái kernel nên vẫn phải trap. Ngoài `getpid`, xv6 có hai lời gọi phù hợp:

- **`sbrk(0)`** chỉ trả `p->sz`. Thêm `uint64 sz` vào `struct usyscall`; kernel ghi `p->usyscall->sz` mỗi khi `p->sz` đổi (`growproc()`/`sys_sbrk` cả nhánh lazy, `exec()`, `fork()` cho con). Chỉ chính process đổi `sz` của nó nên bản sao luôn đúng; `sbrk(0)` chỉ còn là một lệnh load.
- **`uptime()`** chỉ trả biến toàn cục `ticks`. Ghi `ticks` vào trang của từng process mỗi tick tốn $O(\text{NPROC})$; tốt hơn là cấp *một* trang lúc boot, map chỉ đọc (`PTE_R | PTE_U`) ở một địa chỉ cố định khác trong mọi user page table, và `clockintr()` ghi `ticks` vào đó cạnh `ticks++` (như vDSO/vvar của Linux). Load một word 32 bit căn lề thấy giá trị cũ hoặc mới; trễ một tick không tệ hơn system call.

7. Task 5: Use superpages

7.1. Thiết kế

Megapage là PTE lá ở mức 1, PPN trỏ tới đầu một vùng vật lý 2 MB căn 2 MB [4]. Theo gợi ý của đề, `kvmmap()` gọi `kmappages()`, bản sao của `mappages()` chỉ `kvmmap` dùng; `mappages()`, `uvmalloc()`, `uvmcopy()` của user giữ nguyên, nên user page table không có superpage.

- **Chọn kích thước tại từng địa chỉ.** Nếu `a` và `pa` đều căn 2 MB và vùng còn ít nhất 2 MB thì map một lá mức 1 rồi tiến 2 MB, ngược lại map 4 KB. Phần đầu và đuôi không căn của một vùng tự dùng trang 4 KB. `kwalk1()` trả về PTE mức 1 và chỉ cấp trang mức 1; dùng `walk(..., 1)` sẽ cấp thêm một trang mức 0 vô ích.
- **Kiểm tra pa**: PPN của megapage phải căn 2 MB. Direct map có `pa = va` nên điều kiện này trùng với điều kiện của `va` (E6), nhưng giữ cho hàm đúng với mọi lời gọi. **Điều kiện "còn $\geq$ 2 MB"** là bắt buộc: thiếu nó thì trang UART thành megapage và boot panic (E7).
- **Remap.** PTE đã có `V` thì `panic`. `walk()` của starter trả về luôn lá mức 1 khi gặp megapage, nên map 4 KB chồng lên megapage cũng bị bắt, thay vì coi vùng dữ liệu như một trang page table.

```
while (a < end) {
  if ((a % SUPERPGSIZE) == 0 && (pa % SUPERPGSIZE) == 0 && end - a >= SUPERPGSIZE) {
    if ((pte = kwalk1(kpgtbl, a)) == 0)          // level-1 PTE
      return -1;
    if (*pte & PTE_V)
      panic("kmappages: remap");
    *pte = PA2PTE(pa) | perm | PTE_V;             // one 2 MB leaf
    a += SUPERPGSIZE; pa += SUPERPGSIZE;
  } else {
    /* same as mappages(): walk(kpgtbl, a, 1), check, 4 KB leaf */
    a += PGSIZE; pa += PGSIZE;
  }
}
```

kernel/vm.c, vòng lặp của kmappages() (rút gọn)

`vmprint`. `vmprint_level()` đã nhận lá ở mọi mức và không đệ quy vào lá, nên chỉ thêm hậu tố `superpage 2MB` (`1GB` ở mức 2) cho lá ở mức trên 0. `vmprint_test` chỉ in user page table (không có superpage) nên output của nó không đổi; để xem kernel page table, `main()` gọi `vmprint(kernel_pagetable)` sau `kvminithart()` khi build bằng `make clean; make qemu EXTRAFLAG=-DKVMPRINT`. `user/kpgstat.c` in hai số đếm của `ksupernpte()`.

```
page table 0x0000000087fff000
 ..0x0000000000000000: pte 0x0000000021fff801 pa 0x0000000087ffe000
 .. ..0x000000000c000000: pte 0x0000000003000007 pa 0x000000000c000000 RW superpage 2MB
 .. ..0x000000000c200000: pte 0x0000000003080007 pa 0x000000000c200000 RW superpage 2MB
[...]
 .. ..0x000000000fe00000: pte 0x0000000003f80007 pa 0x000000000fe00000 RW superpage 2MB
 .. ..0x0000000010000000: pte 0x0000000021fff401 pa 0x0000000087ffd000
 .. .. ..0x0000000010000000: pte 0x000000000040000c7 pa 0x0000000010000000 RW
 .. .. ..0x0000000010001000: pte 0x0000000004000407 pa 0x0000000010001000 RW
 ..0x0000000080000000: pte 0x0000000021fff001 pa 0x0000000087ffc000
 .. ..0x0000000080000000: pte 0x0000000021ffec01 pa 0x0000000087ffb000
 .. .. ..0x0000000080000000: pte 0x000000002000004b pa 0x0000000080000000 RX
[...]
 .. .. ..0x0000000080007000: pte 0x0000000020001c0b pa 0x0000000080007000 RX
 .. .. ..0x0000000080008000: pte 0x0000000020002047 pa 0x0000000080008000 RW
[...]
 .. .. ..0x00000000801ff000: pte 0x0000000002007fc07 pa 0x00000000801ff000 RW
 .. ..0x0000000080200000: pte 0x0000000020080007 pa 0x0000000080200000 RW superpage 2MB
[...]
 .. ..0x0000000087e00000: pte 0x0000000021f80047 pa 0x0000000087e00000 RW superpage 2MB
[...]
$ kpgstat
kernel page table: 95 superpage PTEs (2 MB), 579 level-0 PTEs (4 KB)
memory mapped: 190 MB by superpages, 2316 KB by 4 KB pages
level-0 page-table pages saved: 95 (380 KB), PTEs saved: 48640
```

Hình 6. Kernel page table do `vmprint` in khi boot với `-DKVMPRINT` (trích, `[...]` là dòng được lược) và output của `kpgstat`.

7.2. Q4. Bao nhiêu PTE superpage? Tiết kiệm bao nhiêu trang mức cuối?

Kernel page table dùng **95 PTE superpage**: 95 dòng `superpage 2MB` trong output của `vmprint`, khớp với `ksupernpte()` (Hình 6).

Vùng (kvmmake)	Quyền	Cách map
PLIC `0x0c000000`–`0x0fffffff` (64 MB)	RW	32 superpage
RAM `0x80200000`–`0x87ffffff` (126 MB)	RW	63 superpage
RAM `0x80000000`–`0x801fffff` (2 MB đầu)	RX, RW	512 trang 4 KB: 8 trang text đến `etext = 0x80008000` rồi 504 trang data (khác quyền; data bắt đầu ở `etext`, không căn 2 MB)
UART0, VIRTIO0, trampoline, 64 kernel stack	RW, RX	67 trang 4 KB

Bảng 5. Kernel page table sau Task 5: 95 superpage và 579 PTE mức 0 (`ksuper_test` cần ≥ 93 và ≤ 1610).

Không có superpage, mỗi vùng 2 MB này cần trọn một trang page table mức 0 (512 PTE). Vậy 95 superpage tiết kiệm **95 trang page table mức cuối** (380 KB) và $95 \times 512 = 48\,640$ PTE: từ 49 219 xuống 579 PTE mức 0, đúng như `kpgstat` in ra khi cho `kvmmap()` dùng lại `mappages()` (E8). Cả kernel page table giảm từ 102 trang (1 gốc, 3 mức 1, 98 mức 0) xuống 7 (1, 3, 3), và mỗi mục TLB của megapage phủ 2 MB thay vì 4 KB.

8. Kiểm thử

Test của `grade-lab-pgtbl`	Nội dung	Sau Task 1–3	Cuối cùng
`pgtbltest: vmprint_test`	11 regex về thụt lề, va và quyền (mục 4)	10/10	10/10
`pgtbltest: pgaccess`	`pgaccess_test: OK`	10/10	10/10
`pgtbltest: ugetpid`	`ugetpid_test: OK`	0/10	10/10
`pgtbltest: superpg`	`ksuper_test: OK` (≥ 93 PTE superpage, ≤ 1610 PTE thường)	0/10	10/10
`answers-pgtbl.txt`	Tệp có nội dung (chấm tay phần còn lại)	5/5	5/5
`usertests`	`usertests -q` in `ALL TESTS PASSED` (test 0 điểm)	đạt	đạt
`time`	`time.txt` chứa một số nguyên	1/1	1/1
Tổng		**26/46**	**46/46**

Bảng 6. Các test của `make grade`. Hai test `ugetpid` và `superpg` thuộc Task 4 và 5.

Sau Task 4 và 5, `make grade` đạt 46/46 (Hình 7); với Task 1 đến 3 là 26/46. `usertests -q` in `ALL TESTS PASSED`, kể cả bước `countfree()` không thấy mất trang; `pgaudit` vẫn đạt 23/23 ca. Mã biên dịch với `-Wall -Werror` không có cảnh báo; cảnh báo `LOAD segment with RWX permissions` là của linker với kernel gốc.

Thí nghiệm phản chứng (Bảng 7): tám bản kernel, mỗi bản đổi đúng một chỗ của Task 4, 5 trên một bản sao của mã, rồi chạy lại test.

```
$ make grade
make clean
riscv64-linux-gnu-ld: warning: kernel/kernel has a LOAD segment with RWX permissions
== Test pgtbltest ==
$ make qemu-gdb
(9.3s)
== Test   pgtbltest: vmprint_test ==
  pgtbltest: vmprint_test: OK
== Test   pgtbltest: pgaccess ==
  pgtbltest: pgaccess: OK
== Test   pgtbltest: ugetpid ==
  pgtbltest: ugetpid: OK
== Test   pgtbltest: superpg ==
  pgtbltest: superpg: OK
== Test answers-pgtbl.txt ==
answers-pgtbl.txt: OK
== Test usertests ==
$ make qemu-gdb
(135.4s)
== Test time ==
time: OK
Score: 46/46
```

Hình 7. Toàn bộ output của `make grade` sau Task 4 và 5 (chạy với `MAKEFLAGS=s` nên không in dòng lệnh build).

	Thay đổi	Kết quả thật và kết luận
E1	Bỏ `kfree(p->usyscall)`	60 test OK rồi `lost some free pages 25902 (out of 32545)`.
E2	Bỏ `uvmunmap(..., USYSCALL, 1, 0)`	`panic: freewalk: leaf` ngay ở `exec("/init")`.
E3	`do_free = 1` cho `USYSCALL`	`ugetpid_test failed: mismatched PID #1`: `exec()` free trang mà page table mới vẫn map.
E4	Bỏ `PTE_U`	`usertrap(): unexpected scause 0xd, stval=0x3fffffd000`: load page fault.
E5	Thêm `PTE_W`	`all tests succeeded`: test không bắt được, nhưng user sửa được pid; quyền quá rộng.
E6	Bỏ điều kiện `pa % SUPERPGSIZE == 0`	Mọi test đạt, vẫn 95 và 579: các vùng ≥ 2 MB của `kvmmake` có `pa = va`.
E7	Bỏ điều kiện `end - a >= SUPERPGSIZE`	`panic: kmappages: remap` lúc boot: trang UART thành megapage phủ cả VIRTIO0.
E8	`kvmmap()` gọi lại `mappages()`	`kpgstat`: 0 superpage, 49 219 PTE mức 0; `ksuper_test failed`.

Bảng 7. Thí nghiệm phản chứng. Mỗi dòng là một bản kernel riêng, chỉ khác bản cuối ở đúng một chỗ.

9. Thảo luận và tự đánh giá

9.1. Khó khăn và cách xử lý

Vấn đề	Nguyên nhân và cách xử lý
Output thật khác ví dụ trong đề (một trang text, data ở `0x1000`, va in đủ 40 bit)	Đọc program header bằng `readelf` và `user/user.ld` để biết trang nào chứa gì, rồi giải thích theo output thật.
Định dạng `vmprint` theo PDF không qua grader	PDF làm mất dấu cách đầu dòng; regex của grader cần `" .."`. Đọc `grade-lab-pgtbl` trước khi in.
`%p` với `uint64` làm build lỗi	`printk` có thuộc tính `format(printf)` và kernel build với `-Werror`; ép đối số sang `void *`.
Buffer kết quả của test chỉ có 4 byte	Ghi đúng `(npages+7)/8` byte, không ghi cả `uint64`.
Con trỏ `malloc` không căn trang; `walk()` panic khi `va >= MAXVA`	Làm tròn `base` xuống đầu trang; kiểm tra cả khoảng trang dưới `MAXVA` trước khi gọi `walk()`.
Test gốc vẫn đạt khi xóa `PTE_A` trước `copyout`	Viết `pgaudit`, xác nhận bằng hai phiên bản kernel sai (mục 5).
Build lỗi trên máy thứ hai	`-ffile-prefix-map` vỡ vì đường dẫn có dấu cách; build trên bản sao trong WSL.
Không thấy được superpage	`vmprint_test` chỉ in user page table; thêm `-DKVMPRINT` và `kpgstat`.

Bảng 8. Các khó khăn gặp phải.

9.2. Hạn chế

- Mỗi lời gọi `pgaccess` quét tối đa 4096 trang (16 MB); chương trình cần quét nhiều hơn phải gọi nhiều lần.
- Mỗi trang được `walk()` hai lần nếu có bit `A` (lượt 1 và lượt 3). Lưu con trỏ PTE để tránh việc này cần 32 KB cho 4096 trang, không đặt được trên kernel stack.
- Trang `sbrklazy` chưa được chạm tới làm `pgaccess` trả -1 (mục 5).
- `USYSCALL` chỉ chứa `pid`. 2 MB đầu của RAM vẫn dùng trang 4 KB (text và data khác quyền).

9.3. Tự đánh giá theo rubric

Hạng mục	Điểm	Mức	Minh chứng
T1. Page table của user (Q1)	1.0	Xuất sắc	20 dòng được giải mã (Bảng 3): nội dung, mọi bit quyền kể cả `A`, `D`; đối chiếu với `readelf`.
T2a. Mã `vmprint`	1.0	Xuất sắc	Đạt `vmprint_test`; bỏ PTE không hợp lệ; đệ quy, nhận lá theo bit quyền.
T2b. Q2	0.5	Xuất sắc	Sáu lá, quyền cho phép và không cho phép, liên hệ với Task 1 (Bảng 4).
T3. `pgaccess`	2.0	Xuất sắc	Đạt `pgaccess`; -1 với trang chưa map và buffer sai; `PTE_A` đúng bit 6, xóa sau khi báo; dùng `copyout`; giới hạn 4096 trang; 23/23 ca của `pgaudit`.
T4a. Trang `USYSCALL`	1.25	Xuất sắc	Đạt `ugetpid`; `R|U`; đủ vòng đời; không mất trang; E1–E5.
T4b. Q3	0.25	Xuất sắc	`sbrk(0)`, `uptime()` và cách kernel cập nhật trang.
T5a. Superpage	2.0	Xuất sắc	Đạt `superpg`; đường map riêng cho kernel; `vmprint` đánh dấu; E6–E8.
T5b. Q4	0.5	Xuất sắc	95 superpage, khớp `vmprint`, `ksupernpte`; tiết kiệm 95 trang (Bảng 5).
Mã, git, `time.txt`	0.5	Xuất sắc	Phong cách xv6, có chú thích; mỗi phần một commit; `time.txt` = 10.
Báo cáo	1.0	Xuất sắc	Đủ các mục của đề, ảnh `make grade` và output, không quá 10 trang.

Bảng 9. Nhóm tự đánh giá theo rubric của đề. Đây không phải điểm đã được giảng viên xác nhận.

10. Tài liệu tham khảo

[1] CS333 Hệ điều hành, *Lab 3: Page Tables* (đề bài, yêu cầu nộp bài và rubric), 2026.

[2] MIT 6.1810, *Lab: page tables*, Fall 2026. https://pdos.csail.mit.edu/6.828/2026/labs/pgtbl.html

[3] R. Cox, M. F. Kaashoek, R. Morris, *xv6: a simple, Unix-like teaching operating system (RISC-V)*, chương 3 (Hình 3.2 và 3.4). https://mit-pdos.github.io/xv6-riscv-book/

[4] *The RISC-V Instruction Set Manual, Volume II: Privileged Architecture*, phần Sv39: định dạng PTE, bit `A/D`, megapage. https://github.com/riscv/riscv-isa-manual

[5] Mã nguồn xv6-labs-2026, nhánh `pgtbl`: `kernel/{vm,exec,proc,kalloc,stats,sysproc}.c`, `kernel/trampoline.S`, `kernel/memlayout.h`, `kernel/riscv.h`, `user/{pgtbltest,umalloc}.c`, `user/user.ld`.

[6] Công cụ AI. Claude Code (Anthropic) được dùng cho Task 1 đến 3: đọc đề và mã nguồn xv6, viết mã `vmprint`, `pgaccess` và chương trình `pgaudit`, chạy thử và chụp ảnh minh chứng, soạn câu trả lời Q1, Q2 và báo cáo. Với Task 4, 5: viết mã, chạy `make grade` và E1–E8, render ảnh từ log, soạn Q3, Q4 và mục 6, 7. Nhóm chịu trách nhiệm về toàn bộ nội dung bài nộp.